Day 14: GPIO & UART with Zephyr

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

gpio_pin_configure_dt with DTS

The Zephyr GPIO API uses devicetree specs to configure pins without hardcoding pin numbers:

```
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);

// Configure as output, initially inactive (LED off)
gpio_pin_configure_dt(&led, GPIO_OUTPUT_INACTIVE);

// Configure as input with pull-up
gpio_pin_configure_dt(&btn, GPIO_INPUT | GPIO_PULL_UP);
```

`GPIO_DT_SPEC_GET` extracts the GPIO controller device, pin number, and flags from the DTS at compile time.

UART IRQ-Driven RX

Polling UART (checking a register in a loop) wastes CPU. **IRQ-driven** UART uses an interrupt to notify when a byte arrives — the CPU can do other work while waiting:

```
uart_irq_callback_user_data_set(uart_dev, my_uart_cb, NULL);
uart_irq_rx_enable(uart_dev);

// ISR callback — called when UART RX FIFO has data
static void my_uart_cb(const struct device *dev, void *user_data) {
    if (!uart_irq_update(dev) || !uart_irq_rx_ready(dev)) return;
    uint8_t ch;
    while (uart_fifo_read(dev, &ch, 1) == 1) {
        // Process ch — store in ring buffer
```

```
    }
}
```

Devicetree Overlays for Custom Boards

An **overlay file** (`.overlay`) extends or overrides the board's default DTS without modifying the board files. Create `boards/nrf52840dk_nrf52840.overlay` :

```
&uart0 {
    current-speed = <115200>;
    status = "okay";
};
&spi1 {
    my_sensor: sensor@0 {
        compatible = "my,sensor";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};
```

DTS Aliases and Chosen Nodes

- **Aliases** (`/aliases`) provide short names for nodes: `led0 = &led_0` → access via `DT_ALIAS(led0)`
- **Chosen** (`/chosen`) selects the default device for a role: `zephyr,console = &uart0` → Zephyr uses uart0 for printk

uart_irq_callback_set

The canonical Zephyr pattern for UART IRQ-driven operation:

1. `uart_irq_callback_user_data_set(dev, cb, ctx)` — register callback with context
2. `uart_irq_rx_enable(dev)` — enable RX interrupt
3. In callback: `uart_irq_update()` → `uart_irq_rx_ready()` → `uart_fifo_read()`

nRF52840 UARTE Peripheral

The nRF52840 has two UART peripherals: **UART0** and **UART1**, plus their enhanced versions **UARTE0** and **UARTE1** (UART with EasyDMA). Zephyr's nrfx UART driver uses UARTE with DMA for efficient

transfers. The nRF52840DK connects UARTE0 to the J-Link USB serial port (appears as `/dev/ttyACM0` on Linux).

Practice Tasks

1. Configure all 4 LEDs on nRF52840DK and implement a binary counter (0-15) driven by a timer
2. Build a UART echo: receive bytes over serial and transmit them back, with local echo
3. Add a button that changes the UART baud rate between 9600 and 115200 at runtime
4. Create a DTS overlay that reassigns UART TX/RX to different pins — verify with a logic analyzer

---

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day14
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach   # RTT viewer (requires J-Link)
```

Resources

- Zephyr Docs
- nRF52840 Product Specification
- Nordic DevZone
- Source: src/day14/main.cpp